

Projekt

SmartHomeHaus (Fa. Keyestudio)



ESPHome ist eine mächtige Plattform, um Mikrocontroller wie ESP8266 oder ESP32 in smarte Home Assistant Geräte zu verwandeln. Diese Anleitung zeigt, wie ESPHome installiert, konfiguriert und integriert wird.

I. Einführung in ESPHome

ESPHome ist ein Open-Source Projekt, mit dem bestimmte Mikrocontroller (z.B. ESP8266 oder ESP32) mittels YAML-Konfigurationsdateien zu individuell angepassten Smart Home Geräten gemacht werden können. Die Konfiguration beschreibt Sensoren, Aktoren, Sensoren und andere Hardware-Komponenten. ESPHome übersetzt diese YAML-Dateien in eine individuelle Firmware, die direkt auf den Mikrocontroller geflasht wird.

II. Vorbereitung: Hardware und Software

Benötigte Hardware

- ESP32 (z.B. ESP32 DevKit) --> Teil des Keyestudio Smart Home
- USB-Kabel (zum Flashen und für die Stromversorgung)
- Optional: Sensoren/Aktoren (z.B. Temperatur-, Feuchtigkeitssensor, ...)

Benötigte Software

- Home Assistant (installiert und laufend)
- ESPHome: App für HomeAssistant
- Powershell zum Flashen (in MS Windows enthalten)
- Editor zum Erstellen / Ändern der YAML-Datei

III. YAML-Datei

Die zu erstellende YAML-Datei soll folgende Benennung erhalten:

`keyestudio_smart_home.yaml`

Code:

Mit dem nachfolgenden YAML-Code sind alle Sensoren und Aktoren im Smart Home von Keyestudio parametrisiert. Die YAML-Datei liegt zum Download im entsprechenden Teams-Kanal zum Labor und beinhaltet folgende Informationen:

- Namensgebung:
`esphome:`
 `name: keyestudio`
 `friendly_name: keyestudio`
- Welcher ESP32 kommt zum Einsatz
`esp32:`
 `board: esp32dev`
 `# variant: esp32s3`
 `framework:`
 `type: esp-idf`
- Wifi-Parameter

```
wifi:
  ssid: "iot"
  password: "smarthome"
```

- Parameter für API (=Application Programming Interface)

```
api:
  #Used for buzzer music
  services:
    - service: play_rtttl
      variables:
        song_str: string
      then:
        - rtttl.play:
            rtttl: !lambda 'return song_str;'
```

- Parameter für OTA (= Over The Air)

```
ota:
  - platform: esphome
```

- Datenlogger

```
logger:
```

- Web Server

```
web_server:
  port: 80
```

- Parameter für Sensoren und Aktoren

```
captive_portal:
```

```
i2c:
  - id: i2c_bus
    frequency: 800kHz
```

```
rc522_i2c:
  address: 0x28
  i2c_id: i2c_bus
```

```
display:
  # LCD Screen
  - platform: lcd_pcf8574
    update_interval: 10s
    id: lcd_display
    dimensions: 16x2
    address: 0x27
    lambda: |-
      it.printf(0, 0, "temp: %.1f C", id(temperature_dht).state);
      it.printf(0, 1, "humid: %.1f %%", id(humidity_dht).state);
```

```
binary_sensor:
  # Reports if this device is Connected or not
```

```

- platform: status
  name: "Status"

# Left push button
- platform: gpio
  pin:
    number: 16
    inverted: true
  name: "Left button"

# Right push button
- platform: gpio
  pin:
    number: 27
    inverted: true
  name: "Right button"

# GAS Sensor
- platform: gpio
  pin:
    number: 23
    inverted: true
  name: "Gas sensor"
  device_class: gas

# PIR Sensor
- platform: gpio
  pin: 14
  name: "PIR sensor"
  device_class: motion

# Blue NFC tag - keyholder
- platform: rc522
  uid: 4A-DC-8B-B6
  name: "blue door chip"

# White NFC tag - credit card
- platform: rc522
  uid: 26-70-85-3D
  name: "white door card"

light:
  # Status LED
  - platform: status_led
    name: "LED"
    pin: 12

  # 6812RGB Led
  - platform: esp32_rmt_led_strip
    rgb_order: GRB

```

```

pin: GPIO26
num_leds: 4
chipset: SK6812
is_rgbw: true
effects:
  - pulse:
  - random:
  - strobe:
  - flicker:
  - addressable_rainbow:
  - addressable_color_wipe:
  - addressable_scan:
  - addressable_twinkle:
  - addressable_random_twinkle:
  - addressable_fireworks:
name: "RGB leds"

servo:
  # Servo 90 Motor 1
  - id: servo_01
    output: servo_01_out

  # Servo 90 Motor 2
  - id: servo_02
    output: servo_02_out

output:
  # Roof FAN
  - platform: ledc
    pin: 18
    frequency: 10000 Hz
    id: ventilation_fan_pwm_r

  # Roof FAN
  - platform: ledc
    pin: 19
    frequency: 10000 Hz
    id: ventilation_fan_pwm_l

  # Servo 90 Motor 1
  - platform: ledc
    id: servo_01_out
    pin: 5
    frequency: 50 Hz

  # Servo 90 Motor 2
  - platform: ledc
    id: servo_02_out
    pin: 13

```

```

    frequency: 50 Hz

# Buzzer config
- platform: ledc
  id: rtttl_out
  pin: 25

# Buzzer config
rtttl:
  output: rtttl_out
  on_finished_playback:
    - logger.log: 'Song ended!'

number:
  # Servo 90 Motor 1
  - platform: template
    name: "window servo"
    min_value: 0
    initial_value: 0
    max_value: 100
    step: 1
    optimistic: true
    set_action:
      then:
        - servo.write:
            id: servo_01
            level: !lambda 'return x / 100.0;'

  # Servo 90 Motor 2
  - platform: template
    name: "door servo"
    min_value: 0
    initial_value: 0
    max_value: 100
    step: 1
    optimistic: true
    set_action:
      then:
        - servo.write:
            id: servo_02
            level: !lambda 'return x / 100.0;'

fan:
  # Roof FAN L
  - platform: speed
    output: ventilation_fan_pwm_1
    speed_count: 100

```

```

    name: "fan left"

# Roof FAN R
- platform: speed
  output: ventilation_fan_pwm_r
  speed_count: 100
  name: "fan right"

sensor:
  # Reports the WiFi signal strength
  - platform: wifi_signal
    name: "WIFI signal"
    update_interval: 60s

  - platform: dht
    pin: 17
    model: dht11
    temperature:
      name: "Temperature"
      id: temperature_dht
    humidity:
      name: "Humidity"
      id: humidity_dht
    update_interval: 30s

  # Reports how long the device has been powered (in minutes)
  - platform: uptime
    name: "up-time"
    filters:
      - lambda: return x / 60.0;
    unit_of_measurement: minutes

# Steam Sensor
- platform: adc
  pin: 34
  name: "rain water"
  update_interval: 10s

switch:
  # Helps to restart the device at distance
  - platform: restart
    name: "restart"

text_sensor:
  # Reports the ESPHome Version with compile date
  - platform: version
    name: "ESPHome Version"

# Reports WiFi name

```

```
- platform: wifi_info
  ssid:
    name: "WiFi"

# Reports current IP
- platform: wifi_info
  ip_address:
    name: "IP-Adress"
```

IV. Flashen des ESP32 über Powershell

Voraussetzungen:

- Git muss installiert sein
- Python (3.13) muss installiert sein
(Kontrolle in der Powershell über „py --version“)

Erstellen einer virtuellen Umgebung:

- Ordner `keyestudio` auf C: erstellen
- In der Powershell (wenn möglich als Administrator starten) den zuvor erstellten Ordner öffnen
- mit `py -3.13 -m venv keyestudio-venv` eine neue virtuelle Umgebung erstellen
- mit `.\keyestudio-venv\Scripts\Activate.ps1` die virtuelle Umgebung aktivieren

HINWEIS: Wenn eine Fehlermeldung bzgl. Deaktivierung zur Ausführung von Skripts kommen sollte, dann kann die Ausführung mit folgendem Befehl in der Powershell aktiviert werden:

`Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`

--> mit „A“ für [A] Ja, alle bestätigen

--> sollte Pkt. 5 notwendig sein, dann Pkt. 4 nochmals wiederholen

- die virtuelle Umgebung ist aktiv, wenn vor dem Ordner in grüner Schrift der Text `(keyestudio-venv)` steht
- mit `pip install esphome` ESPHome installieren

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Alle Rechte vorbehalten.

Installieren Sie die neueste PowerShell für neue Funktionen und Verbesserungen!
https://aka.ms/PSWindows

PS C:\Users\Thomas> cd\
PS C:\> cd hocht_keyestudio
PS C:\hocht_keyestudio> py --version
Python 3.13.12
PS C:\hocht_keyestudio> py -3.13 -m venv hocht_keyestudio-venv
PS C:\hocht_keyestudio> .\hocht_keyestudio-venv\Scripts\Activate.ps1
(hocht_keyestudio-venv) PS C:\hocht_keyestudio> pip install esphome
```


- mit `esphome --version` kann überprüft werden, ob ESPHome installiert ist

```
(hocht_keyestudio-venv) PS C:\hocht_keyestudio> esphome --version
Version: 2026.3.1
```

- die erstellte YAML-Datei muss im erstellten Ordner `keyestudio` und nicht im virtuellen Ordner `keyestudio_venv` liegen!
- die YAML-Datei in der Powershell über den Befehl `esphome run .\keyestudio_smart_home.yaml` auf den ESP flashen

```
(hocht_keyestudio-venv) PS C:\hocht_keyestudio> esphome run .\hocht_keyestudio_smart_home.yaml
```

Am Ende muss noch der zugehörige COM-Port ausgewählt werden:

```
===== [SUCCESS] Took 277.36 seconds =====
INFO Build Info: config_hash=0x060c70b8 build_time_str=2026-03-26 19:04:28 +0100
INFO Successfully compiled program.
Found multiple options for uploading, please choose one:
  [1] COM12 (USB-SERIAL CH340 (COM12))
  [2] COM4 (Seriellles USB-Gerät (COM4))
  [3] COM5 (Seriellles USB-Gerät (COM5))
  [4] Over The Air (hocht_keyestudio.local)
(number):
```

HINWEIS: Der ESP könnte auch direkt in Homeassistant über die App [ESPHome Builder](#) geflasht werden, allerdings funktioniert das nicht immer, daher die Empfehlung des Flashens über die PowerShell

- Kontrolle ob Flashen funktioniert hat:

```
[10:01:00.151][I][wifi:1551]: Connected
[10:01:00.154][C][wifi:1221]: IP Address: 192.168.2.207
[10:01:00.159][C][wifi:1232]: SSID: 'iot'
[10:01:00.162][C][wifi:1232]: BSSID: 70:A7:41:F1:D2:96
[10:01:00.165][C][wifi:1232]: Hostname: 'hocht_keyestudio'
[10:01:00.170][C][wifi:1232]: Signal strength: -60 dB
[10:01:00.170][C][wifi:1232]: Channel: 6
[10:01:00.173][C][wifi:1232]: Subnet: 255.255.255.0
[10:01:00.173][C][wifi:1232]: Gateway: 192.168.2.1
[10:01:00.176][C][wifi:1232]: DNS1: 192.168.2.1
[10:01:00.179][C][wifi:1232]: DNS2: 0.0.0.0
[10:01:00.182][W][component:429]: wifi cleared Warning flag
[10:01:55.355][I][safe_mode:091]: Boot seems successful; resetting boot loop counter
[10:01:58.277][D][esp32.preferences:144]: Writing 1 items: 0 cached, 1 written, 0 failed
```

Nicht sicher 192.168.2.207

HTLVB

Lehrplan_BAU

Privat

HomeAssistant

PV / EEG / BEG

H2-Demoanlage

Fuel Cell

Arduino

ChatBot

ID Austria verwalten

hoct_keystudio

Name	State	Actions
ESPHome Version	2026.3.1 (config hash 0fce86cd34, built 2026-03-25 10:49:46 +0100)	
Gas sensor	OFF	
Humidity	28 %	
IP-Address	192.168.2.207	
LED	OFF	Off ☐ On ☐
Left button	OFF	
PIR sensor	OFF	
RGB leds	OFF	Off ☐ On ☐ None 0 255
Right button	OFF	
Status	ON	
Temperature	25.5 °C	
WiFi signal	-51 dBm	
WiFi	iot	
blue door chip	OFF	
door servo	0	0 100
fan left	OFF	70 Off ☐ On ☐ 0 100
fan right	OFF	100 Off ☐ On ☐ 0 100
rain water	0.08 V	
restart	OFF	Off ☐ On ☐
up-time	8 minutes	
white door card	OFF	
window servo	0	0 100

Scheme

OTA Update

Date auswählen Keine Datei ausgewählt Update

Time	Level	Tag	Message
12:53:31	[D]	[sensor:124]	'humidity' >> 26 %
12:53:40	[D]	[sensor:124]	'rain water' >> 0.08 V
12:53:51	[D]	[sensor:124]	'rain water' >> 0.08 V
12:54:00	[D]	[sensor:124]	'WiFi signal' >> -51 dBm
12:54:00	[D]	[sensor:124]	'rain water' >> 0.08 V
12:54:01	[D]	[dht:044]	Temperature 25.5°C humidity 25.0%
12:54:01	[D]	[sensor:124]	'Temperature' >> 25.5 °C
12:54:01	[D]	[sensor:124]	'humidity' >> 25 %
12:54:03	[D]	[sensor:124]	'up-time' >> 6 minutes
12:54:10	[D]	[sensor:124]	'rain water' >> 0.08 V
12:54:20	[D]	[sensor:124]	'rain water' >> 0.08 V
12:54:30	[D]	[sensor:124]	'rain water' >> 0.08 V
12:54:33	[D]	[dht:044]	Temperature 25.5°C humidity 28.0%
12:54:33	[D]	[sensor:124]	'Temperature' >> 25.5 °C
12:54:33	[D]	[sensor:124]	'humidity' >> 28 %
12:54:40	[D]	[sensor:124]	'rain water' >> 0.08 V
12:54:51	[D]	[sensor:124]	'rain water' >> 0.08 V
12:55:00	[D]	[sensor:124]	'WiFi signal' >> -50 dBm
12:55:00	[D]	[sensor:124]	'rain water' >> 0.08 V
12:55:01	[D]	[dht:044]	Temperature 25.5°C humidity 25.0%
12:55:01	[D]	[sensor:124]	'Temperature' >> 25.5 °C
12:55:01	[D]	[sensor:124]	'humidity' >> 25 %
12:55:10	[D]	[sensor:124]	'up-time' >> 7 minutes
12:55:10	[D]	[sensor:124]	'rain water' >> 0.08 V
12:55:20	[D]	[sensor:124]	'rain water' >> 0.08 V
12:55:46	[D]	[sensor:124]	'rain water' >> 0.08 V
12:55:46	[D]	[dht:044]	Temperature 25.5°C humidity 25.0%
12:55:46	[D]	[sensor:124]	'Temperature' >> 25.5 °C
12:55:46	[D]	[sensor:124]	'humidity' >> 25 %
12:55:46	[D]	[sensor:124]	'rain water' >> 0.08 V
12:55:50	[D]	[sensor:124]	'rain water' >> 0.08 V
12:55:50	[D]	[sensor:124]	'WiFi signal' >> -51 dBm
12:55:50	[D]	[sensor:124]	'rain water' >> 0.08 V
12:55:50	[D]	[dht:044]	Temperature 25.5°C humidity 25.0%
12:55:50	[D]	[sensor:124]	'Temperature' >> 25.5 °C
12:55:50	[D]	[sensor:124]	'humidity' >> 25 %
12:55:50	[D]	[sensor:124]	'up-time' >> 8 minutes
12:55:12	[D]	[sensor:124]	'rain water' >> 0.08 V
12:56:21	[D]	[sensor:124]	'rain water' >> 0.08 V
12:56:22	[D]	[sensor:124]	'rain water' >> 0.08 V
12:56:42	[D]	[dht:044]	Temperature 25.5°C humidity 25.0%
12:56:42	[D]	[sensor:124]	'Temperature' >> 25.5 °C

V. Integration ESPHome in Home Assistant

- Integration ESPHome in HomeAssistant unter Einstellungen / Geräte / Integration hinzufügen

✕ Anbieter auswählen

Nach einem Markennamen suchen

esphome

✕



ESPHome

>

- Den entsprechenden Host (= IP-Adresse des geflashten ESP32) eintragen

✕ ESPHome



Bitte gib die Verbindungseinstellungen deines ESPHome-Geräts ein.

Host*

Erforderlich

Port

6053

Port, auf dem die native API läuft

OK

- Nach der Installation der Integration sollte alle Sensoren und Aktoren des Keystudio Smart Home in HomeAssistant aufscheinen und auch schaltbar sein. Damit können nun viele unterschiedliche Automatisationen realisiert werden.

